

Automatic Modulation Recognition: An FPGA Implementation

Satish Kumar¹, Graduate Student Member, IEEE, Rajarshi Mahapatra², Senior Member, IEEE,
and Anurag Singh³, Member, IEEE

Abstract—Very recently, convolution neural network (CNN) based deep-learning (DL) models have been used in automatic modulation classification (AMC) systems and achieved superior performance. However, the huge CNN model size and floating-point weights and activation make deployment of such systems very complex with limited hardware resources such as FPGAs. In this work, we have designed CNN-based AMC schemes for the complex-valued temporal radio signal domain and implemented them on an FPGA platform. This work focuses on quantized CNN and low precision mathematics to solve the problem of bigger model size and floating-point weights and activations. However, the low precision weights, activation, and quantized CNN significantly impact the model's accuracy. To overcome this, we proposed an RU-based AMC scheme and used an iterative pruning-based training mechanism to maintain the overall accuracy above a certain threshold while decreasing the model size on hardware. The proposed scheme achieves at least 1.4% higher accuracy and takes only 40% hardware resources than the baseline. The RFSoc implemented model achieves a real-time throughput of 527k classification per second with a latency of 7.5μs.

Index Terms—Automatic modulation classification, convolution neural network, iterative pruning, quantization, FPGA.

I. INTRODUCTION

RECENT advancements in convolutional neural network (CNN) based deep learning (DL) techniques reduce the difficulties of automatic modulation classification (AMC) while increasing its accuracy. Without prior knowing the signal power, offset in frequency and phase, timing error, and carrier frequency, identification of the modulation scheme remains challenging [1]–[3]. The statistical machine learning (ML) approaches and the maximum likelihood-based theoretical decision strategy are both used to find the modulation scheme [1], [2]. Maximum likelihood approaches require prior knowledge of noise and the received signal's actual probability density functions (PDFs). Not only that, but phase jitter, erroneous channel state information, frequency offset etc., all significantly impact maximum likelihood approaches. However, the ML approach, uses feature extraction to classify patterns [1], [2]. It is not easy to choose features with high

discriminant qualities. ML-based classifiers' performance suffers from feature selection errors. Recently found DL models with huge CNN layers show the remarkable capacity of feature learning on raw multidimensional data with supervised objectives. Despite the amazing performance of DL networks due to the very high computing and storage requirements of convolutional and fully connected layers [4]–[5], implementation of DL networks on power and resource-constrained hardware such as FPGAs and RFSocs is still far-fetched. These difficulties in real-time hardware implementation motivate us to provide the strategies to optimize the DL networks so that the DL-based AMC methods can be implemented on resource-constrained FPGAs.

Compressed DL networks that exploit efficient computation and quantization of DL networks with low-precision mathematics are two famous methods to reduce the complexity and storage requirement of DL-based schemes. Iandola *et al.* [4] replaced the larger convolutional kernels with smaller kernels that further reduced the AlexNet parameters by 50 times while keeping the accuracy nearly same. Liu *et al.* [5] proposed Sparse CNN to reduce the inherent redundancy. To provide efficiency in hardware implementation, many quantization approaches for neural networks have been investigated [6]. Under low-precision weights and activations, state-of-the-art accuracy has been achieved [7]. Numerous accelerator architectural concepts have been proposed for low-precision CNNs with uniform quantization arithmetic [8]. Other FPGA architectures have been developed to use CNNs' highly adaptable nature, limiting weights to binary or ternary representations [9].

O'Shea *et al.* [3] developed and tested two CNN models (VGG10 and ResNet33), demonstrating competitive accuracy over 24 modulation classes. Runtime boundaries for real-time implementation are not provided or are incomplete in all of these studies, which is a research gap that we hope to address in this work. This letter proposes two DL-based AMC schemes and implements them in real-time on RFSoc ZCU111 using quantized DL and low precision arithmetics. In particular, The main contributions of this work are:

- We have proposed two less complex compressed CNN-based models for AMC.
- A low precision fully quantized weight and activation-based CNN implementation have been used to further reduce the memory and resource requirement.
- An RU-based AMC scheme has been proposed and iterative pruning-based training method has been used to increase the classification accuracy while making the proposed network sparse.

Manuscript received 25 March 2022; revised 19 May 2022; accepted 14 June 2022. Date of publication 23 June 2022; date of current version 11 September 2022. The authors acknowledge the financial assistance from the UGC, New Delhi, in the form of the NET- Junior Research Fellowship (UGC NET-JRF), which supports the work. The associate editor coordinating the review of this letter and approving it for publication was L. Fu. (Corresponding author: Satish Kumar.)

The authors are with the Department of Electronics and Communication Engineering, Dr. S. P. M. International Institute of Information Technology, Naya Raipur, Raipur, Chhattisgarh 493661, India (e-mail: satishkumarvatsa@gmail.com).

Digital Object Identifier 10.1109/LCOMM.2022.3184771

1558-2558 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

- Implement the proposed AMC schemes in real-time on the RFSoc ZCU111 hardware platform.

II. SYSTEM MODEL

A. Neural Network on FPGAs

CNN is computationally expensive in terms of inference and training. Also, we must note that current models of CNNs are overly parameterized, which opens up the opportunity of reducing the precision bit with the loss of some or even no accuracy. Generally, during training of DL models, single or double floating-point mathematics are used, the reduced precision and pruning can be used during testing such models without a significant decrease in accuracy. This leads to new opportunities for building accelerators for FPGA deployment. In recent years, FPGAs have been a preferred platform for implementing quantized CNNs [7], [10]. For CNNs to be implemented on FPGAs, they must occupy less resource and memory space. To compress DL networks, smaller kernel sizes and layers with a small number of output channels are being used. In addition, quantized CNN is a popular choice. We can apply quantization to weight and activations [6], [7]. Weights are generally 32-bit floating points that are usually downgraded to 8-bit or even lower, which helps in reducing the size of the model. We can even quantize a floating point to an integer. Quantization reduces the model's size considerably at the trade-off of some accuracy, so a fair number of precision bits must be chosen carefully. For the implementation of quantized neural network and quantization aware training (QAT), Brevitas [11] with Pytorch in python has been used.

B. Convolutional Layer Based Network (QMCNet)

The proposed AMC models are designed efficiently according to the requirement of the hardware implementation of such models. Now onwards, we call this model as QMCNet. The proposed QMCNet architecture uses 6 one dimensional (1D) convolution (conv1D) layers with smaller kernel sizes and fewer output channels to the CNN layers. This compressed architecture makes QMCNet less complex with a low memory footprint while providing comparable accuracy than the baseline (VGG10 network in [3]). The overall architecture of the proposed QMCNet is displayed in Table I. This QMCNet achieves only 1.2% accuracy degradation with 27.3% less multiply-and-accumulate bits operations (*bit_operation*) than the baseline. To further make our model less memory greedy, inputs, weights, and activations have been quantized. On the one side, using quantization saves the memory while reducing the accuracy on the other side. To maintain good accuracy, after many trials, we choose inputs, activations, and weights to be represented by 4, 5, and 6 bits, respectively. This quantized QMCNet has only 1.17%, 7.30%, and 38.60% *bit_operations*, weight bits, and activation bits, respectively, than the unquantized baseline while it shows 3.75% degradation in accuracy.

C. Residual Unit Based Network (RUNet)

In the previous sub-section, we have discussed a simple CNN based AMC scheme to lower the complexity. However,

TABLE I
QMCNet NETWORK LAYERS

Layer	Output Dimension
Input	2×1024
Conv	16×1024
Maxpool	16×512
Conv	24×512
BatchNormalization	24×512
Maxpool	24×256
Conv	32×256
BatchNormalization	32×256
Maxpool	32×128
Conv	48×128
BatchNormalization	48×128
Maxpool	48×64
Conv	64×64
BatchNormalization	64×64
Maxpool	64×32
Conv	96×32
BatchNormalization	96×32
Avgpool	96×1
FullyConnected/ReLu	24
FullyConnected/Softmax	24

the QMCNet shows 3.75% accuracy degradation. To increase the accuracy, we further utilize deeper networks. With a deeper network, while training at first, accuracy gets saturated and then degrades rapidly. The over-fitting of the network is not the cause of the above degradation problem. With improvement in DL techniques, training of deeper networks are now possible. In [12], authors have shown that compared to swallow non-residual networks, residual networks can gain accuracy from the considerably increased depth and are easier to optimize. Following this approach, a residual-unit (RU) has been designed, which can gain accuracy from the considerably increased depth and are easier to optimize [12].

The residual unit (RU) and a composite convolution block (ConvB), containing multiple Conv1D layers, play a significant role in extracting the in-depth features from the received raw input signal. In RU, skip connections are used to operate the strong discriminant features at multiple depths and scales throughout the network. Fig. 1a shows the internal structure of RU. It consists of two one-dimensional convolutions (Conv1D) layers followed by a batch normalization layer and a ReLu activation function. A skip connection is made between the input and output of this layer arrangement to make it residual and prevent the network from overfitting during the training process. For more details about the residual network, readers are advised to read [12]. The main difference between RU and ConvB block is that in ConvB block, two Conv1D layers with the different kernels, stride, and output channel sizes have been used to extract the vertical features on the spatial dimension in a better fashion. Fig. 1b and fig. 1c represent the layering structure of convB block and proposed RU-based proposed RUNet for AMC. In Fig.1, O , O_1 , and O_2 represent the number of output channels, K , $Num_classes$ represent the kernel size and the total modulation classes respectively, S represents the stride of the respective layer, and FC represents a fully connected or dense layer. After many trials, we choose inputs, weights, and activations to be represented by 6-bit quantization to maintain accuracy.

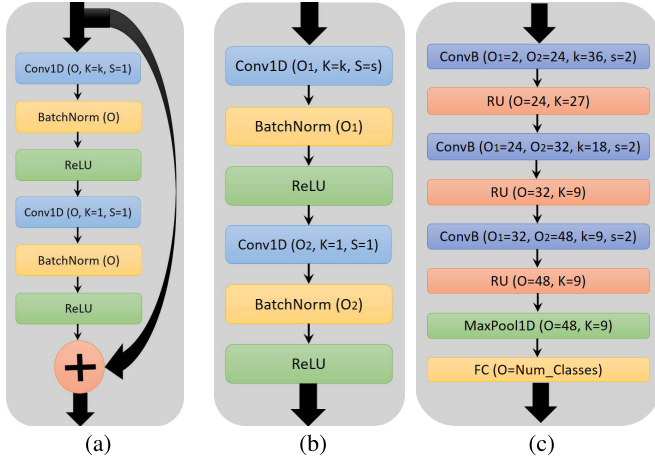


Fig. 1. (a) Internal architecture of a Residual Unit (RU), (b) Internal architecture of a convolutional block (ConvB), and (c) Proposed RU-based AMC system model (RUNet).

D. Training and Pruning Methodology

Quantized CNN and compressed network make the proposed QMCNet and RU-based AMC less complex and offer small memory footprints. Still, the large number of trainable parameters hinders its hardware implementation. Identifying and eliminating these parameters helps reduce model complexity without compromising performance. To address this issue and further make the proposed models less complex iterative pruning methodology has been used while training the networks. Pruning reduces the number of parameters and, as a result, the computational complexity of the provided model by removing fully connected neurons with negligible weight. Algorithm 1 represents the iterative pruning methodology used to train the networks, where minimum achievable accuracy is set as 58%. Note that this value can be easily changed based on the requirement but within the limitations of the architecture performance. After successive pruning and retraining, it is seen that the weights those are having near-zero values are converted to zero, and retraining distributes the contribution of those weights to remaining non-zero weights. Fig. 2 represents the weights distribution after successive pruning and iteration(s). Each successive pruning increases the number of zero-valued weights and distributes its contribution to all other non-zero weights.

III. SIMULATION RESULTS

The Open-source RadioML 2018.01A dataset has been used to train and test our DL-based AMC schemes. This RadioML 2018.01A dataset has been generated in real-time using USRP devices in natural environments as described in [3]. For details of the RadioML dataset, readers are advised to read [3]. In this section, we present the performance of the proposed AMC systems. The proposed systems have been evaluated based on classification accuracy over a wide range of SNRs and complexity. For clarity, in this letter, we represent the (Input, Activation, Weight) quantization bits along with the model name in brackets. Also superscript ^p in (Input, Activation, Weight)^p represents that the model is pruned.

Algorithm 1 Iterative Pruning Based Training Method

Input: Total number of pruning iteration $k = 50$, Pruning Amount per iteration $\delta = 0.2$, Total number of training Epochs = $E = 30$, Minimum Accuracy = 58%

Output: Pruned model

```

for Pruning Iterations=1:k do
  Best Accuracy = 0
  for Epochs =1:E do
    Train the Model and find Test Accuracy
    if Test Accuracy  $\geq$  Best Accuracy then
      save model Weights
      Best Accuracy = Test Accuracy
    end
    if Test Accuracy < Best Accuracy for 10 consecutive Epochs then
      Stop Training
    end
  end
  if Test Accuracy < Minimum Accuracy then
    Stop Pruning Iteration
  end
  Prune the model weight to  $\delta$ 
end

```

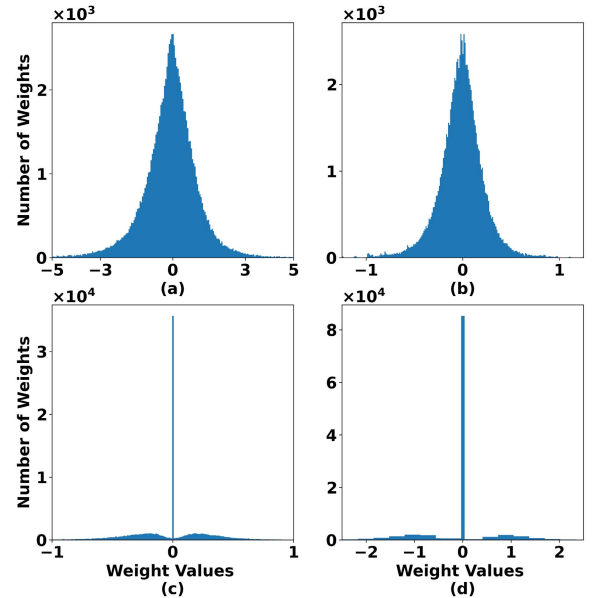


Fig. 2. Weight distribution of QMCNet (a) Unquantized unpruned model (b) Quantized unpruned model (c) Quantized pruned model with 1 iteration (d) Quantized pruned model with 10 iterations.

A. Classification Performance

The proposed AMC schemes have been evaluated using software simulation^a as well as hardware implementation. Fig. 3 represents the average classification accuracy of all 24 modulation schemes vs. SNRs. From this figure following observations were made:

^aThe code for this work is available at: https://github.com/satishkumarvatsa/AMC_on_FPGA

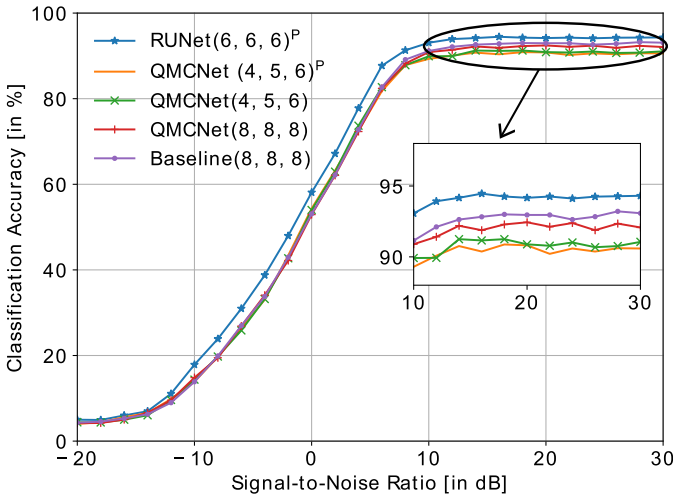


Fig. 3. Overall accuracy of all modulation schemes averaged over SNR ranges $[-20, +30]$ dB for CNN model.

- 1) The classification accuracy of the proposed schemes and the baseline(8,8,8) is more than 50% above 0 dB SNR.
- 2) The accuracy of the RUNet(6,6,6)^P scheme is at least 1.4% higher than baseline(8,8,8) above -12 dB SNR.
- 3) The accuracy of the proposed QMCNet(8,8,8) and baseline(8,8,8) are comparable throughout the SNR range.
- 4) The proposed QMCNet(4,5,6)^P scheme provides similar accuracy to baseline(8,8,8) up to 8 dB SNR. Above +8 dB, the baseline(8,8,8) accuracy is higher.
- 5) The accuracy of the QMCNet(4,5,6)^P and QMCNet(4,5,6) are nearly the same. This indicates that pruning has little impact on classification accuracy.
- 6) The maximum accuracy provided by RUNet(6,6,6)^P network, QMCNet(4,5,6)^P and baseline(8,8,8) are 94.46%, 90.58% and 93.07% respectively at +30 dB SNR.

The confusion matrix for the QMCNet(4,5,6)^P is shown in Fig.4a for all 24 classes at +30dB SNR. High-order quadrature amplitude modulation (QAM) (64/128/256-QAM) and AM modulations are the biggest sources of errors. Large inaccuracy is expected for short-time observations with significant noise, especially with higher-order QAM, due to a lack of information and similar symbol structure. The confusion matrix for the RUNet(6,6,6)^P is shown in Fig.4b for all 24 classes at +30dB SNR. This RUNet technique reduces the classification error among high order QAM (64/128/256-QAM) and attains higher accuracy than QMCNet(4,5,6)^P by 3.88% at +30dB SNR.

B. Model Complexity

The higher computational complexity of any DL model hinders its implementation in any area and power-constrained FPGA hardware. For analyzing the complexity of our proposed AMC models, the following methodology has been used:

- The complexity calculation will take into account the number of multiply-accumulate (MAC) operations and the number of parameters for convolutional (Conv), matrix multiplication, and general matrix multiplication operations, assuming a naive/direct implementation.

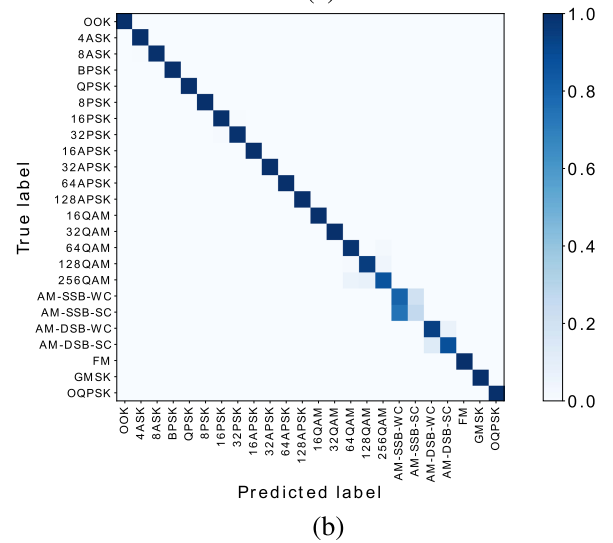
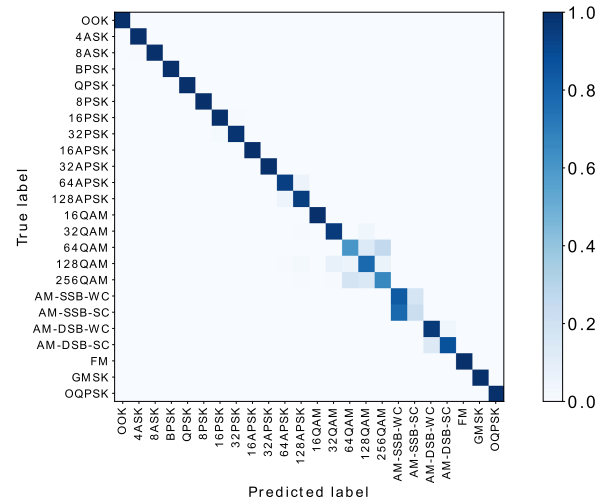


Fig. 4. (a)Confusion matrix for QMCNet(4,5,6)^P (b)Confusion matrix for RUNet(6,6,6)^P for all 24-modulations at +30dB SNR.

- The element-wise nonlinearities, pooling, batch normalization, Mul/Div, and Add/Sub for applying biases or quantization scaling factors have little impact on the total model complexity and are assumed to be zero-cost.
- The complexity cost will discount for parameter sparsity by $discounted_MAC = MAC * (num_nonzero_ops/num_total_ops)$ and $discounted_mem = mem * (num_nonzero_params/num_total_params)$ for each layer. Activation sparsity and any other form of dynamic sparsity is not taken into account.
- The complexity cost will account for precision as follows: $bit_operations = discounted_MAC * weight_bits * act_bits$ and $bit_mem = discounted_mem * weight_bits$ for each layer. Here $bit_operations$ represents the total bit operations required by the network, $weight_bits$ represents the total bits required to store weights and act_bits represents the total bits required to store activations.

Table II represents the calculated complexity cost of the proposed AMC schemes with baseline. Here, “Total act_bits ”,

TABLE II
COMPLEXITY COST AND ACCURACY COMPARISON

Model	Precision in bits (Input, Weight, Activation)	Total bit_operations	Total act_bits	Total weight_bits	Accuracy at +30 dB SNR
QMCNet	unquantized	9578.28m	1.71m	3.14m	93.50%
QMCNet	(8, 8, 8)	487.24m	1.71m	0.629m	92.07%
QMCNet	(4, 5, 6)	154.63m	1.61m	0.372m	90.96%
QMCNet	(4, 5, 6) ^p	64.47m	1.61m	0.154m	90.59%
RUNet	(6, 6, 6)	74.59m	2.92m	0.044m	94.64%
RUNet	(6, 6, 6) ^p	37.45m	2.92m	0.022m	94.46%
Baseline [3]	(8, 8, 8)	807.69m	4.17m	1.24m	93.07%
Baseline [3]	unquantized	13173.26m	4.17m	5.09m	94.70%

TABLE III
RESOURCE UTILIZATION ON ZCU111 RFSoc

Model	CLBs	LUTs	FFs	BRAMs	DSPs
QMCNet (4,5,6) ^p	19358	61364	40476	57	0
RUNet (6,6,6) ^p	11345	34563	21357	40	0
Baseline (8,8,8)	28000	85151	54483	70	0

“Total *weight_bits*” and “Total *bit_operations*” represent the total number of activation bits, the total number of weight bits, and total multiply-accumulate bit operations respectively. This table shows that the proposed QMCNet(4,5,6)^p model has only 0.49% bit_operations and 3.03% weight bits than unquantized baseline whereas the RUNet(6,6,6)^p model has only 0.28% bit_operation and 0.44% weight bits than unquantized baseline. Whereas the maximum likelihood-based traditional AMC method shows exponential complexity with modulation order and the number of antennas. Apart from that, we have implemented the proposed AMC systems in real time on the RFSoc platform, the details of which are presented in the next section.

IV. HARDWARE IMPLEMENTATION

Vivado 2020.1 tool has been used to synthesize the proposed schemes for hardware implementation. At first, the Brevitas [11] with PyTorch libraries was used to train and test the proposed schemes on the GPU platform. The training of the QMCNet scheme took 19 Hrs. and 13 min, and the RUNet scheme took 25 Hrs. and 27 min on NVIDIA GT1080Ti GPUs. Many element-wise operations with low arithmetic intensity were involved during QAT. Further, this contributes to a more involved computational graph during the backpropagation operation. Due to this, the QAT becomes more resource greedy and slower than standard floating-point training. By exporting the network to FINN [10] via a custom ONNX-based representation, the network may be mapped to a low-precision integer-only dataflow accelerator on a Xilinx FPGA. Convolutional, fully connected, pooling, and LSTM layer types are implemented as streaming components using FINN’s HLS hardware library. For details of FINN and streaming components implementation, readers are advised to read [10] and references. In this work, FINN with Vivado 2020.1 is used to create HLS IP per network layer, and then stitched IP design and PYNQ runtime driver are generated for the specific RFSoc ZCU111 board.

A. Resource Utilization

The resource consumption for the various models evaluated in this work is shown in Table III. Because the convolution operations do not require any DSPs, the DSP utilization is relatively consistent. The QMCNet scheme uses 69% CLBs and 72% LUTs, whereas RUNet (6,6,6) scheme uses 40% CLBs and 39% LUTs than baseline (8,8,8).

V. CONCLUSION

This letter demonstrates the real-time implementation of DL-based AMC models on a RFSoc. In this, quantized CNN, QAT and iterative pruning methodologies have been used to reduce the models’ overall complexity and memory requirement while keeping the average accuracy over a certain threshold. The proposed QMCNet(4,5,6)^p has 91% lesser bit_operations, 61.4% lesser activation bits, and 87.5% lesser weight bit, while the RUNet(6,6,6)^p model has 95.37% fewer bit_operations, 30% lesser activation bits, and 98.22% lesser weight bits than the baseline(8,8,8) model. On the ZCU111 platform, RUNet achieves a competitive 94.46% maximum accuracy on the 24-class RadioML dataset at +30dB SNR while delivering a high throughput of 527K classifications/s and a classification latency of just 7.5μs.

REFERENCES

- [1] M. A. Abdel-Moneim, W. El-Shafai, N. Abdel-Salam, E. M. El-Rabaie, and F. E. Abd El-Samie, “A survey of traditional and advanced automatic modulation classification techniques, challenges, and some novel trends,” *Int. J. Commun. Syst.*, vol. 34, no. 10, p. e4762, Jul. 2021.
- [2] T. Huynh-The *et al.*, “Automatic modulation classification: A deep architecture survey,” *IEEE Access*, vol. 9, pp. 142950–142971, 2021.
- [3] T. J. O’Shea, T. Roy, and T. C. Clancy, “Over-the-air deep learning based radio signal classification,” *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 168–179, Feb. 2018, doi: 10.1109/JSTSP.2018.2797022.
- [4] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size,” 2016, *arXiv:1602.07360*.
- [5] B. Min, W. Foroosh, H. Tappen, M. Pensky, and M. Sparse, “Convolutional neural networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Boston, MA, USA, Jun. 2015, pp. 7–12.
- [6] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” 2021, *arXiv:2103.13630*.
- [7] S. Tridgell, D. Bolland, P. H. W. Leong, R. Kastner, A. Khodamoradi, and Siddhartha, “Real-time automatic modulation classification using RFSoc,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, May 2020, pp. 82–89.
- [8] C. Wu, M. Wang, X. Chu, K. Wang, and L. He, “Low precision floating-point arithmetic for high performance FPGA-based CNN acceleration,” 2020, *arXiv:2003.03852*.
- [9] D. J. Moss *et al.*, “A customizable matrix multiplication framework for the Intel HARPv2 Xeon+FPGA platform: A deep learning case study,” in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, New York, NY, USA, 2018, pp. 107–116, doi: 10.1145/3174243.3174258.
- [10] Y. Umuroglu *et al.*, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, New York, NY, USA: ACM, 2017, pp. 65–74.
- [11] A. Pappalardo. (2021). *Xilinx/Brevitas*. Zenodo Publisher. [Online]. Available: <https://doi.org/10.5281/zenodo.3333552>
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.